

# 5. Forecasting Framework and Baselines

## The Day-Ahead Forecasting Problem

### What Exactly Are We Predicting?

Before building any model, we must define the forecasting problem with absolute precision. Vagueness at this stage — "predict electricity prices" — leads to models that look good on paper but are useless in practice. Here is the exact specification:

- **Target variable:** The 30-minute **trading price** (\$/MWh) for a single NEM region (SA1 in this project), transformed via arcsinh (see Chapter 1).
- **Forecast horizon:** 48 half-hour periods, covering one full trading day (from 04:00 to 04:00 AEST the following day).
- **Gate closure:** 12:00 noon today. The forecast must be issued using *\*only\** information available before noon. No data from 12:01 onwards is allowed.
- **Available information at gate closure:** All historical prices, demand, weather, and AEMO pre-dispatch data up to (but not including) noon.
- **Output format:** Either a **point forecast** (a single predicted price for each half-hour) or a **probabilistic forecast** (predicted quantiles or a full distribution for each half-hour).

**Gate closure:** The deadline by which a forecast must be issued. In our day-ahead setup, gate closure is noon — the forecast for the next trading day must be finalised by 12:00. After gate closure, no additional information can be incorporated into the forecast. This mirrors the operational reality of a battery operator who must commit to a dispatch plan before the trading day begins.

**Forecast horizon:** The time span into the future that a forecast covers. A 48-step horizon at 30-minute resolution covers 24 hours. The difficulty of forecasting generally increases with horizon — predicting the price one hour from now is much easier than predicting the price 20 hours from now, because more can change in a longer period.

**Point forecast:** A single predicted value for each future time step — one number per half-hour. A point forecast says "the price at 6pm will be \$85." It gives the model's best guess but conveys no information about how confident the model is.

**Probabilistic forecast:** A prediction that describes a range of possible outcomes and their likelihoods — for example, quantiles (10th, 50th, 90th percentile) or a full probability distribution. A probabilistic forecast says "the price at 6pm has a 10% chance of being below \$40, a 50% chance of being below \$85, and a 10% chance of exceeding \$250." This additional information is critical for optimal battery dispatch under uncertainty.

This is a **multi-step** forecasting problem: we need 48 values, not one. Each of the 48 predictions has a different effective horizon — the first prediction is for ~16 hours ahead (noon to 4am), and the last is for ~40 hours ahead (noon to 4am the following day). Error generally grows with horizon, so later predictions are less reliable.

### Why Day-Ahead?

The day-ahead horizon is not an academic choice — it matches the **decision cycle** of a grid-scale battery operator:

1. Before noon, the operator receives a price forecast for the next 24-hour trading day.
2. Based on this forecast, they solve an optimisation problem: when should the battery charge? When should it discharge? When should it sit idle?
3. The resulting dispatch schedule is executed against actual spot prices.
4. Revenue =  $\Sigma$  (price  $\times$  net discharge) over all half-hours.

**Trading day:** The NEM's trading day runs from 04:00 AEST to 04:00 AEST the following day — not midnight to midnight. This convention exists because the overnight period (midnight to 4am) has consistent low-demand, low-price characteristics that belong logically with the preceding day. All 48 half-hours of a trading day are settled together.

**Real-world example — the battery operator's morning:** A battery operator at the Hornsdale Power Reserve (the "Tesla Big Battery" in South Australia) might receive a price forecast at 11:30am showing: low prices overnight (\$20–40), a deep solar dip to ~\$10 at midday, and an evening spike to \$200 at 6pm. The optimal strategy is clear: charge during the solar dip (buy cheap), discharge during the evening spike (sell expensive). But the operator must commit to this plan before noon — if the forecast is wrong and the spike does not materialise, the battery discharges into a \$50 market instead of a \$200 market, earning a fraction of the expected revenue.

Other horizons serve different purposes:

| Horizon               | Decision supported                  | Data requirements                         |
|-----------------------|-------------------------------------|-------------------------------------------|
| Intra-day (1–4 hours) | Real-time dispatch adjustments      | Real-time data feeds, rapid model updates |
| Day-ahead (24 hours)  | Daily dispatch planning (our focus) | Morning data, noon gate closure           |

|                    |                                          |                                       |
|--------------------|------------------------------------------|---------------------------------------|
| Week-ahead         | Maintenance scheduling, contract hedging | Less accurate but useful for planning |
| Month/season-ahead | Capacity investment, long-term contracts | Highly uncertain, scenario-based      |

## The Target Transform in Practice

Recall from Chapter 1 that electricity prices have extreme statistical properties — a range from −\$1,000 to \$17,500 with heavy tails and asymmetry. We model the **arcsinh-transformed** price, not the raw price. The complete forecast workflow is:

1. Transform:  $y = \text{arcsinh}(\text{price})$  2. Train model to predict  $y$  3. Generate forecast:  $y^*$  4. Invert:  $\text{predicted\_price} = \sinh(y^*)$  5. Evaluate:  $\text{MAE}(\text{actual\_price}, \text{predicted\_price})$

**The evaluation rule:** Always train in arcsinh space but evaluate in dollar space. If you measure forecast error in arcsinh space, you systematically underweight errors on extreme prices — exactly the events that drive battery revenue. A model that perfectly predicts quiet \$50 prices but misses every \$5,000 spike would look excellent in arcsinh space but would be worthless for dispatch.

## The Feature Matrix

### What Goes Into a Price Forecast?

A forecasting model needs **features** (also called **predictors**, **inputs**, or **independent variables**) — quantities that carry information about future prices. For electricity price forecasting, features fall into five categories:

**Feature (predictor variable):** A measurable quantity used as input to a forecasting model. Features are the "clues" the model uses to predict the target variable (price). Good features have a strong, stable relationship with the target. Bad features add noise without adding information, which can actually make predictions worse.

#### 1. Lagged prices (autoregressive features):

From Chapter 2, we know that electricity prices have strong autocorrelation — today's price at 2pm is strongly related to yesterday's price at 2pm. The most important lags are:

- **Lag 48:** Same half-hour yesterday (24 hours ago at 30-minute resolution). This is the single most powerful feature.
- **Lag 96:** Same half-hour two days ago. Adds information beyond yesterday but with diminishing returns.
- **Lag 336:** Same half-hour one week ago ( $7 \times 48 = 336$ ). Captures the weekly cycle — weekdays differ from weekends.

**Autoregressive feature:** A feature constructed from past values of the target variable itself. "Autoregressive" literally means "self-regressing" — the variable is used to predict itself. The lag-48 price feature says: "use the price 24 hours ago to help predict the price now." Autoregressive features exploit the temporal persistence (autocorrelation) in the data.

#### 2. Calendar features:

The time-of-day, day-of-week, and month encode the systematic patterns in electricity demand and supply:

- **Half-hour of day (0–47):** Captures the diurnal cycle — the duck curve from Chapter 2.
- **Day of week (0–6):** Captures the weekly pattern — weekday demand is higher than weekend demand.
- **Month (1–12):** Captures seasonal variation — summer prices differ from winter prices due to demand and renewable output patterns.
- **Public holiday indicator (0/1):** Holidays have weekend-like demand patterns even on weekdays.

#### 3. Demand features:

- **Lagged demand:** Recent demand levels provide information about the current state of the system.
- **Demand forecasts:** If AEMO's demand forecast for the target period is available (via pre-dispatch), it directly proxies future net load.

#### 4. Weather features:

From Chapter 4, we use:

- **Clear-sky index:** Normalised solar irradiance — proxy for solar generation.
- **Wind speed:** Computed from ERA5  $u_{100}/v_{100}$  — proxy for wind generation.
- **Temperature deviation:** Difference from the 18°C comfort baseline — proxy for heating/cooling demand.

#### 5. Derived (engineered) features:

- **Rolling volatility:** Recent absolute price returns, measuring how turbulent the market has been. Volatility clusters (Chapter 2), so recent volatility predicts near-future volatility.
- **Net load:** Demand minus renewable generation. This single feature encodes the supply–demand balance that drives the hockey-stick price response (Chapter 3).

**Net load:** Total electricity demand minus variable renewable generation (wind + solar). Net load is the amount of electricity that must be supplied by dispatchable generators — coal, gas, hydro, and batteries. It is the single most important driver of electricity prices because it determines which generator is marginal (Chapter 1) and where the system sits on the supply stack (Chapter 3).

## Feature Engineering Principles

**Four rules for feature engineering:** 1. **No future information:** Every feature must be computable from data available *before* gate closure. Using tomorrow's actual temperature as a feature is data leakage — the model would appear to have skill it does not actually possess. 2. **Transform for linearity:** Raw features often have nonlinear relationships with price. The clear-sky index (rather than raw GHI) and  $\text{arcsinh}(\text{price})$  (rather than raw price) improve the linearity of feature–target relationships, which helps linear models and does not hurt nonlinear ones. 3. **Lag for availability:** If a data source has a publication delay, the lag must account for it. If demand data is published with a 1-hour delay, use demand at lag 2 (or earlier), not lag 1. 4. **Encode domain knowledge:** Net load (demand minus renewables) is more informative than demand and renewables as separate features, because it directly corresponds to the supply-stack mechanism that sets the price. Creating this interaction feature saves the model from having to discover it from data.

## Data Leakage: The Silent Model Killer

### What Is Data Leakage?

**Data leakage** occurs when information that would not be available at prediction time is inadvertently included in the training data or features. It causes a model to appear much better during development than it actually performs in production.

**Data leakage:** Any situation where information from the future (relative to the forecast point) contaminates the model's inputs — either during training or prediction. Leakage makes backtested performance overly optimistic because the model is effectively "seeing the answer" before making its prediction. Leakage is insidious because it produces no error messages — the model trains normally, evaluates normally, and gives results that look excellent. The problem only appears when the model is deployed in production and performs much worse than expected.

Leakage is the most dangerous error in time-series forecasting because it is **invisible**. There is no error message, no warning, no obvious sign. The model trains without complaint, produces impressive backtested results, and only fails when deployed in the real world — where it suddenly lacks the future information it was trained on.

### Common Sources of Leakage in Electricity Price Forecasting

- 1. Timestamp misalignment (Chapter 1).** AEMO uses interval-ending timestamps. If you join price data (interval-ending) with weather data (instantaneous) without shifting, you give the model weather measured at the *end* of the price interval — information that was not available when the interval started.
- 2. Using contemporaneous features.** Including tomorrow's actual temperature, wind speed, or demand as a feature when forecasting tomorrow's price. These values are not known at gate closure.
- 3. Training on test data.** If the model's training set overlaps with the test set (either directly or through lagged features that reach into the test period), it has seen the answers before being evaluated.
- 4. Feature scaling on the full dataset.** Computing the mean and standard deviation for feature normalisation using *all* data (including the test set), then normalising both training and test data. The model's scaling parameters now encode information about the test period.
- 5. Rolling statistics that look ahead.** Computing a "rolling mean" centred on each observation (using values both before and after) rather than a trailing rolling mean (using only past values).

**Real-world example — the leakage trap:** A researcher builds a price forecasting model using ERA5 temperature as a feature. They use the temperature at the same timestamp as the price they are predicting — 3pm temperature to predict 3pm price. The model achieves outstanding results: MAE of \$8, far better than any published benchmark. But the 3pm temperature at the time of the 3pm price observation is the *actual* temperature at 3pm — information that would not have been available at noon gate closure. In reality, the model would need to use a temperature *forecast* or a lagged actual value. When deployed with actual weather forecasts (which have errors), the model's MAE jumps to \$25 — still decent, but far from the backtested \$8. The \$17 gap is entirely due to leakage.

**The leakage test:** For every feature, ask: "Could I actually compute this value at gate closure in a live production system?" If the answer is no — if it requires data that would not yet exist — the feature is leaking future information. Apply this test to every single feature in your model. One leaked feature is enough to invalidate your entire backtest.

## The Rolling-Origin Backtest

### Why Not a Simple Train/Test Split?

The most basic evaluation approach is to split data into a training set (first 80%) and a test set (last 20%). Train the model once, predict the test set, compute the error metric. Done.

This is insufficient for electricity price forecasting for two reasons:

**Sample dependence.** A single test period captures only one set of market conditions. Your model might look great on a mild autumn test period but fail completely on a volatile summer period. The results are hostage to the specific dates chosen.

**Non-stationarity.** Electricity prices change over time — the generation mix evolves, fuel prices shift, new solar farms are built, policies change. A model that works well in 2023 Q4 might fail in 2024 Q1. A single split gives one snapshot, not a picture of how well the model adapts.

The **rolling-origin backtest** solves both problems by evaluating the model across *many* train/test splits over time.

## The Rolling-Origin Procedure

**Rolling-origin backtest (walk-forward validation, time-series cross-validation):** An evaluation procedure that simulates the process of repeatedly training and forecasting over time. At each step, the model is trained on all data up to a certain point (the "origin"), makes a forecast for the next  $H$  periods, and the origin is advanced. This produces a sequence of out-of-sample forecasts that spans a long test period, giving a robust estimate of model performance under varying conditions.

The procedure works as follows:

1. **Set the initial origin** at time  $t_1$ . The training set is  $[t_0, t_1]$ .
2. **Apply the embargo.** Skip the next  $H$  periods after the origin (explained below).
3. **Forecast.** Predict the  $H$  periods immediately after the embargo:  $[t_1 + \text{embargo}, t_1 + \text{embargo} + H]$ .
4. **Record errors.** Compare forecasts to actual prices and store the errors.
5. **Advance the origin** by one step (or by `refit_every` steps for computational efficiency).
6. **Expand (or slide) the training window** to include the new data.
7. **Refit the model** on the updated training set.
8. **Repeat** until the origin reaches the end of the available data.

### Rolling-origin backtest with embargo

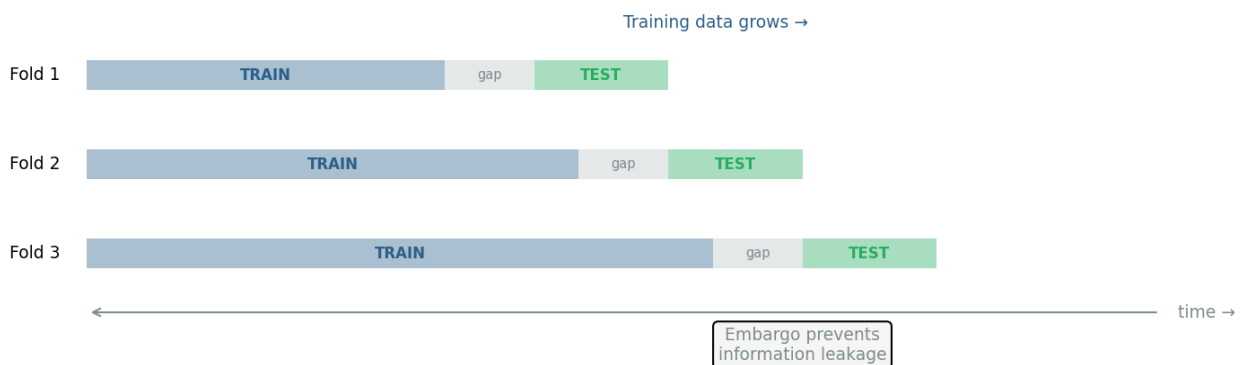


Figure 5.1 — The rolling-origin backtest. The training window (blue) expands as the origin advances. The embargo gap (grey) separates the training data from the forecast period (red) to prevent information leakage through lagged features. Each origin produces one set of 48 half-hourly forecasts.

## The Embargo: Why It Exists

The **embargo** is a gap of excluded data between the end of the training set and the start of the forecast period. It is the single most important anti-leakage mechanism in the backtest.

**Embargo:** A mandatory gap between the end of the training data and the start of the forecast evaluation period. The embargo prevents information leakage that can occur through lagged features. If the model uses a lag-48 feature (price 24 hours ago) and the forecast starts at time  $t+1$ , then the lag-48 feature at time  $t+48$  would reference the price at time  $t$  — which is in the training set. An embargo of at least  $H$  periods (the forecast horizon) ensures that no lagged feature in the forecast period can reference data from the training set.

Here is the leakage scenario the embargo prevents:

- Training data ends at time  $t$ .
- Without embargo, the forecast starts at  $t+1$  and covers periods  $t+1$  through  $t+48$ .
- At forecast period  $t+48$ , the lag-48 feature = price at time  $t+48-48$  = price at time  $t$ .
- Time  $t$  is the *last* observation in the training set — so the model was trained on this value.
- The model's forecast for period  $t+48$  effectively uses a training observation as a feature, creating circular dependency.

With an embargo of  $H = 48$  periods:

- Training data ends at time  $t$ .
- Embargo: periods  $t+1$  through  $t+48$  are excluded (neither trained on nor forecasted).

- Forecast: periods  $t+49$  through  $t+96$ .
- At forecast period  $t+96$ , the lag-48 feature = price at time  $t+48$  — which is in the embargo (not trained on). No leakage.

**The embargo rule:** The embargo must be at least as long as the longest lag used as a feature. If you use lag-336 (one week), the embargo must be at least 336 periods. In practice, setting the embargo equal to the forecast horizon  $H$  is a safe minimum that prevents leakage through the standard lags (48, 96, 336 are all  $\leq 336$ , and  $H$  is typically 48 in day-ahead forecasting). When using longer lags, increase the embargo accordingly.

## Expanding vs. Sliding Windows

The rolling-origin procedure requires choosing how the training window evolves as the origin advances:

**Expanding window:** The training window starts at a fixed point in time and grows longer as the origin advances. Every historical observation is always included. This maximises the amount of training data but may be hurt by non-stationarity — old data from a fundamentally different market structure may mislead the model.

**Sliding window:** The training window has a fixed length (e.g., 2 years). As the origin advances, the window slides forward — old observations are dropped as new ones are added. This adapts to changing conditions but uses less data and may lose useful patterns (e.g., rare seasonal events that occurred outside the window).

| Approach  | Pros                                                      | Cons                                             | Best when                  |
|-----------|-----------------------------------------------------------|--------------------------------------------------|----------------------------|
| Expanding | Maximum data, captures rare events                        | Old data may be misleading if market has changed | Market structure is stable |
| Sliding   | Adapts to structural change, recent data is most relevant | Less data, may miss seasonal patterns            | Market is evolving rapidly |

In the NEM, where the generation mix has changed dramatically (massive solar build-out since 2018, coal plant closures, growing battery fleet), a **sliding window of 2–3 years** often outperforms an expanding window. Data from 2017 reflects a fundamentally different supply stack than 2024, and including it can actively mislead the model.

**Real-world example — the expanding window trap:** Consider training a model for SA1 in 2024. With an expanding window, the training set includes 2018–2023. But in 2018, SA1 had ~1 GW of rooftop solar and negligible utility-scale solar. By 2024, installed solar has tripled. The 2018 data teaches the model that midday prices are moderate (coal sets the price), but in 2024, midday prices are routinely zero or negative (solar surplus sets the price). The old data dilutes the model's ability to learn the current market dynamics.

## Baseline Models: The Bar Your Model Must Clear

### Why Baselines Matter

A forecasting model's value is measured not by its absolute accuracy but by its accuracy **relative to baselines**. A model with an MAE of \$15/MWh sounds impressive — until you learn that a trivial baseline achieves \$16/MWh. The model adds enormous complexity for negligible improvement.

**Baseline model (benchmark):** A simple, often parameter-free model that provides a reference point for evaluating more sophisticated approaches. A model is only considered useful if it outperforms the baseline by a statistically significant margin. In electricity price forecasting, the standard baselines are the "naive" forecast (price at the same time last week) and the AEMO pre-dispatch price.

### The Similar-Day Naive Baseline

The simplest reasonable price forecast exploits the strongest feature in the data — the lag-48 autocorrelation:

$$y_t^* = y_{t-336}$$

This says: the predicted price at time  $t$  is the *actual* price at the same half-hour, same day of the week, one week ago (336 half-hours = 7 days). This is called the **similar-day naive** because it uses the most recent "similar" day as the forecast.

**Similar-day naive forecast:** A forecast that sets each half-hourly price equal to the actual price at the same half-hour on the same day of the week, one week earlier. It captures the diurnal cycle (same half-hour) and the weekly cycle (same day of week) with zero parameters to estimate. Despite its simplicity, it is surprisingly difficult to beat — many published forecasting models that claim to outperform "naive" baselines actually compare against weaker baselines (like a constant-mean forecast).

The similar-day naive captures: - The **diurnal cycle** (same half-hour of day) - The **weekly cycle** (same day of week) - The **current price level** (recent prices, not historical averages)

It does *not* capture: - Weather changes (next week's weather may differ from this week's) - Generator outages or returns - Demand trends (growing or declining load) - Any form of conditional prediction (it makes the same forecast regardless of current conditions)

**Warning — the "naive" trap:** Many academic papers claim their model "beats the naive baseline" but use a weak naive — yesterday's price, or the historical average. The similar-day naive (same day of week, one week ago) is a much harder benchmark because it exploits both the lag-48 and lag-336 autocorrelations, which are the two strongest signals in the data. Always verify which naive baseline a paper uses before being impressed by its results.

## The Autoregressive Baseline

An autoregressive (AR) model uses a **linear combination** of lagged values to predict the future:

**Autoregressive (AR) model:** A model that predicts the current value of a time series as a weighted sum of its own past values (lags) plus a constant. The number of lags included is called the "order" of the AR model. An AR model is equivalent to a linear regression where the features are past values of the target variable.

$$y_t^* = \beta_0 + \beta_1 \cdot y_{t-48} + \beta_2 \cdot y_{t-96} + \beta_3 \cdot y_{t-336} + \epsilon_t$$

where: -  $\beta_0$  is the intercept (a constant baseline level) -  $\beta_1, \beta_2, \beta_3$  are learned coefficients — the model discovers the optimal weight on each lag -  $\epsilon_t$  is the error term

The AR model is the natural next step from the naive baseline. Instead of using lag-48 alone with an implicit coefficient of 1, it **learns the optimal weights** on multiple lags. For example, the model might learn  $\beta_1 = 0.6$ ,  $\beta_2 = 0.15$ ,  $\beta_3 = 0.2$ , meaning yesterday's price gets 60% weight, two days ago gets 15%, and last week gets 20%.

The AR model captures: - The diurnal cycle (lag 48) - The two-day pattern (lag 96) — useful for weekend-to-weekday transitions - The weekly cycle (lag 336) - The **relative importance** of each lag (learned coefficients, not hard-coded)

It does *not* capture: - Nonlinear relationships (it is strictly linear) - Weather effects (no weather features) - Volatility clustering (it assumes the error variance is constant)

## Calendar-Enhanced AR

Adding **indicator variables** for the half-hour of day produces a more flexible baseline:

$$y_t^* = \beta_0 + \sum_k \beta_k \cdot y_{t-lagk} + \sum_h \gamma_h \cdot I(\text{halfhour} = h) + \epsilon_t$$

**Indicator variable (dummy variable):** A binary (0/1) feature that encodes category membership. For half-hour of day, there are 47 indicator variables (one is omitted to avoid perfect multicollinearity):  $I(\text{halfhour}=1) = 1$  if the observation is in half-hour 1, 0 otherwise;  $I(\text{halfhour}=2) = 1$  if the observation is in half-hour 2, and so on. Each indicator allows the model to learn a different average price level for that time period.

The half-hour dummies allow the model to learn that 6pm is systematically more expensive than 3am, independent of what yesterday's prices were. This is valuable because the duck curve shape (Chapter 2) persists across days even when the price level shifts — tomorrow's duck curve may be \$30 higher or lower than today's, but it will still *look* like a duck.

## Forecast Error Metrics

### Mean Absolute Error (MAE)

**Mean Absolute Error (MAE):** The average of the absolute differences between forecasts and actual values. MAE treats positive errors (over-prediction) and negative errors (under-prediction) equally, and treats all observations equally regardless of magnitude. It is measured in the same units as the target variable (\$/MWh for electricity prices).

$$\text{MAE} = (1/N) \times \sum_{t=1}^N |y_t - y_t^*|$$

Example: If the actual prices for three periods are [\$50, \$120, \$30] and the forecasts are [\$55, \$100, \$35]:

- Errors:  $|50-55| = 5$ ,  $|120-100| = 20$ ,  $|30-35| = 5$
- $\text{MAE} = (5 + 20 + 5) / 3 = \$10/\text{MWh}$

MAE is the standard metric for electricity price forecasting because it is **robust to outliers**. Unlike squared-error metrics, a single \$15,000 spike does not dominate the entire evaluation.

### Root Mean Square Error (RMSE)

**Root Mean Square Error (RMSE):** The square root of the average of squared forecast errors. By squaring the errors before averaging, RMSE penalises large errors disproportionately — a single error of \$100 contributes as much to RMSE as ten errors of ~\$32. RMSE is always greater than or equal to MAE, and the gap between them indicates how much the errors vary in magnitude.

$$\text{RMSE} = \sqrt{(1/N) \times \sum_{t=1}^N (y_t - y_t^*)^2}$$

For electricity prices, RMSE is problematic because a few spike events can dominate the metric. A model that perfectly predicts 99% of prices but misses one \$10,000 spike by \$5,000 will have a terrible RMSE, even if its MAE is excellent. This instability makes RMSE unreliable for comparing models on electricity price data.



## Relative MAE (rMAE)

**Relative MAE (rMAE):** The ratio of a model's MAE to the MAE of a baseline (typically the similar-day naive).  $rMAE < 1$  means the model outperforms the baseline;  $rMAE > 1$  means it is worse. rMAE normalises away the absolute price level, making results comparable across different time periods (volatile summers have higher absolute MAE than mild autumns) and regions.

$$rMAE = MAE(model) / MAE(naive\ baseline)$$

rMAE is the preferred comparison metric because it answers the only question that matters: **does this model add value beyond the simplest reasonable alternative?** An rMAE of 0.85 means the model reduces error by 15% relative to the naive — a meaningful improvement. An rMAE of 0.98 means the model barely beats the naive — probably not worth the complexity.

**Real-world example — why rMAE matters:** Model A achieves  $MAE = \$12/MWh$  on a volatile summer test period where the naive achieves  $MAE = \$18$  ( $rMAE = 0.67$ ). Model B achieves  $MAE = \$8/MWh$  on a mild autumn test period where the naive achieves  $MAE = \$10$  ( $rMAE = 0.80$ ). Comparing raw MAEs, Model B looks better (\$8 vs. \$12). But rMAE tells the true story: Model A has more skill relative to its baseline (0.67 vs. 0.80). Model A is producing more *useful* forecasts — it improves most in the periods that are hardest to predict.

## The Diebold-Mariano Test

When comparing two models, simply comparing their MAEs is not enough. The difference might be due to **chance** — random variation in which model happens to be closer to the truth on any given day. We need a formal statistical test to determine whether the difference is **statistically significant**.

**Diebold-Mariano (DM) test:** A statistical hypothesis test that determines whether the difference in forecast accuracy between two models is statistically significant. The test accounts for the fact that forecast errors are typically autocorrelated (today's error is correlated with tomorrow's), which would invalidate simpler comparison methods. A DM p-value below 0.05 provides evidence that the performance difference is real, not just noise.

The DM test works as follows:

1. For each time period  $t$ , compute the **loss differential**:  $d_t = |e_{1,t}| - |e_{2,t}|$ , where  $e_1$  and  $e_2$  are the errors of the two models.
2. Test whether the mean of the  $d_t$  series is significantly different from zero.
3. Use a **HAC (heteroscedasticity and autocorrelation consistent)** standard error estimator to account for the autocorrelation in the  $d_t$  series.

**HAC standard error:** A method for estimating the standard error of a statistic that accounts for both heteroscedasticity (changing variance over time) and autocorrelation (serial dependence) in the data. Standard (OLS) standard errors assume independent, identically distributed observations — an assumption that is always violated in time-series data. HAC standard errors correct for this, producing valid inference even when errors are correlated over time.

**Statistical significance:** A result is statistically significant if it is unlikely to have occurred by chance alone. The conventional threshold is a p-value of 0.05, meaning there is less than a 5% probability of observing a difference at least as large as the one found, if the two models truly had equal performance. Statistical significance does not imply practical significance — a statistically significant improvement of \$0.50/MWh might not justify the cost of a more complex model.

**Why the DM test matters:** In electricity price forecasting, test sets are often short — a few months of data. With short test sets, random variation can easily produce MAE differences of 5–10% between models that are genuinely equal in skill. The DM test prevents you from declaring a spurious "improvement" based on a lucky test period. Never report model comparisons without a DM test (or similar statistical test).

## Battery Dispatch: Converting Forecasts to Revenue

### The Arbitrage Problem

A grid-scale battery earns money through **temporal arbitrage** — buying electricity when it is cheap and selling it when it is expensive:

**Temporal arbitrage:** The practice of exploiting price differences across time. A battery charges (buys electricity from the grid) during low-price periods and discharges (sells electricity back to the grid) during high-price periods. The profit is the price difference minus efficiency losses. Unlike spatial arbitrage (buying in one location, selling in another), temporal arbitrage requires energy storage.

- **Charge** when prices are low: the battery draws electricity from the grid, converting electrical energy to chemical energy. It pays the spot price for this electricity.
- **Discharge** when prices are high: the battery releases its stored energy back to the grid, converting chemical energy to electrical energy. It receives the spot price for this electricity.

The profit per charge–discharge cycle depends on three things:

1. **The price spread:** How much higher the discharge price is than the charge price.
2. **The round-trip efficiency:** Not all energy stored can be recovered — some is lost as heat. Typical lithium-ion batteries achieve 85–92% round-trip efficiency.
3. **The energy volume:** How many MWh are charged and discharged.

**Round-trip efficiency ( $\eta$ ):** The fraction of energy put into a battery that can be recovered when the battery is discharged. If a battery is charged with 100 MWh and can discharge 90 MWh, its round-trip efficiency is 90%. The lost 10 MWh is converted to heat. This means the discharge price must be at least  $\text{charge\_price} / \eta$  (e.g.,  $\text{charge\_price} / 0.9 \approx 1.11 \times \text{charge\_price}$ ) for the cycle to break even.

$$\text{Profit} \approx (P_{\text{discharge}} - P_{\text{charge}} / \eta) \times \text{Energy} \times \Delta t$$

**Real-world example — a battery arbitrage cycle:** Consider the Mannum BESS, a 100 MW / 200 MWh lithium iron phosphate battery in SA1, owned by Epic Energy and optimised by Habitat Energy (it can discharge at 100 MW for 2 hours). During the midday solar dip, the price is \$10/MWh. The battery charges for 2 hours at 100 MW, storing 200 MWh. Cost =  $200 \times \$10 = \$2,000$ . During the evening spike, the price is \$200/MWh. With 90% round-trip efficiency, the battery can discharge 180 MWh. Revenue =  $180 \times \$200 = \$36,000$ . Profit =  $\$36,000 - \$2,000 = \$34,000$  in a single day. On extreme days with \$5,000+ spikes, a single cycle can earn over \$1 million. This is why battery operators care intensely about price forecasts — the forecast determines when to charge and discharge.

## The Linear Program (LP)

Given a price forecast for  $H = 48$  half-hour periods, the **optimal** charge–discharge schedule can be found by solving a **linear program**:

**Linear program (LP):** A mathematical optimisation problem where the objective function (what you want to maximise or minimise) and all constraints are linear — they involve only addition, subtraction, and multiplication by constants, with no squares, exponentials, or other nonlinear operations. LPs can be solved exactly and efficiently using well-established algorithms (simplex method, interior-point methods). The solution is guaranteed to be the global optimum — there is no risk of finding a "local" optimum that is not the best possible answer.

The battery dispatch LP is:

**Objective — maximise revenue:**

$$\text{Maximise: } \sum_{t=1}^H \text{price}_t \times (\text{discharge}_t - \text{charge}_t) \times \Delta t$$

**Subject to constraints:**

$0 \leq \text{charge}_t \leq P_{\text{max}}$  (power limit on charging)  $0 \leq \text{discharge}_t \leq P_{\text{max}}$  (power limit on discharging)  $0 \leq \text{SOC}_t \leq E_{\text{max}}$  (energy capacity limits)  
 $\text{SOC}_{t+1} = \text{SOC}_t + \text{charge}_t \times \sqrt{\eta} \times \Delta t - \text{discharge}_t / \sqrt{\eta} \times \Delta t$  (energy balance)  $\sum \text{discharge}_t \times \Delta t \leq C \times E_{\text{max}}$  (cycle limit)  $\text{SOC}_0 = \text{SOC}_{\text{initial}}$  (initial state)

**State of charge (SOC):** The amount of energy currently stored in the battery, expressed either in MWh or as a percentage of total capacity. SOC evolves over time as the battery charges and discharges. The constraints  $0 \leq \text{SOC} \leq E_{\text{max}}$  prevent over-charging (which damages the battery) and over-discharging (which degrades battery life).

**P\_max (power rating):** The maximum rate at which the battery can charge or discharge, measured in megawatts (MW). A 100 MW battery can absorb or release up to 100 MW of power at any moment. The power rating determines how quickly the battery can fill or empty.

**E\_max (energy capacity):** The total amount of energy the battery can store when fully charged, measured in megawatt-hours (MWh). A 200 MWh battery can store 200 MWh. The ratio  $E_{\text{max}} / P_{\text{max}}$  gives the "duration" — how long the battery can discharge at full power. A 100 MW / 200 MWh battery has a 2-hour duration.

**Cycle limit:** A constraint on how many full charge–discharge cycles the battery can perform per day, imposed to limit degradation. Lithium-ion batteries degrade faster with more cycling. A typical limit is 1–2 full cycles per day. The constraint  $\sum \text{discharge} \times \Delta t \leq C \times E_{\text{max}}$  limits the total discharged energy to  $C$  times the capacity per optimisation window.

The LP splits the round-trip efficiency ( $\eta$ ) into two square-root components: one applied during charging ( $\sqrt{\eta}$ ) and one during discharging ( $1/\sqrt{\eta}$ ). This accurately models the physics — losses occur on both the charge and discharge sides — and keeps the formulation linear.



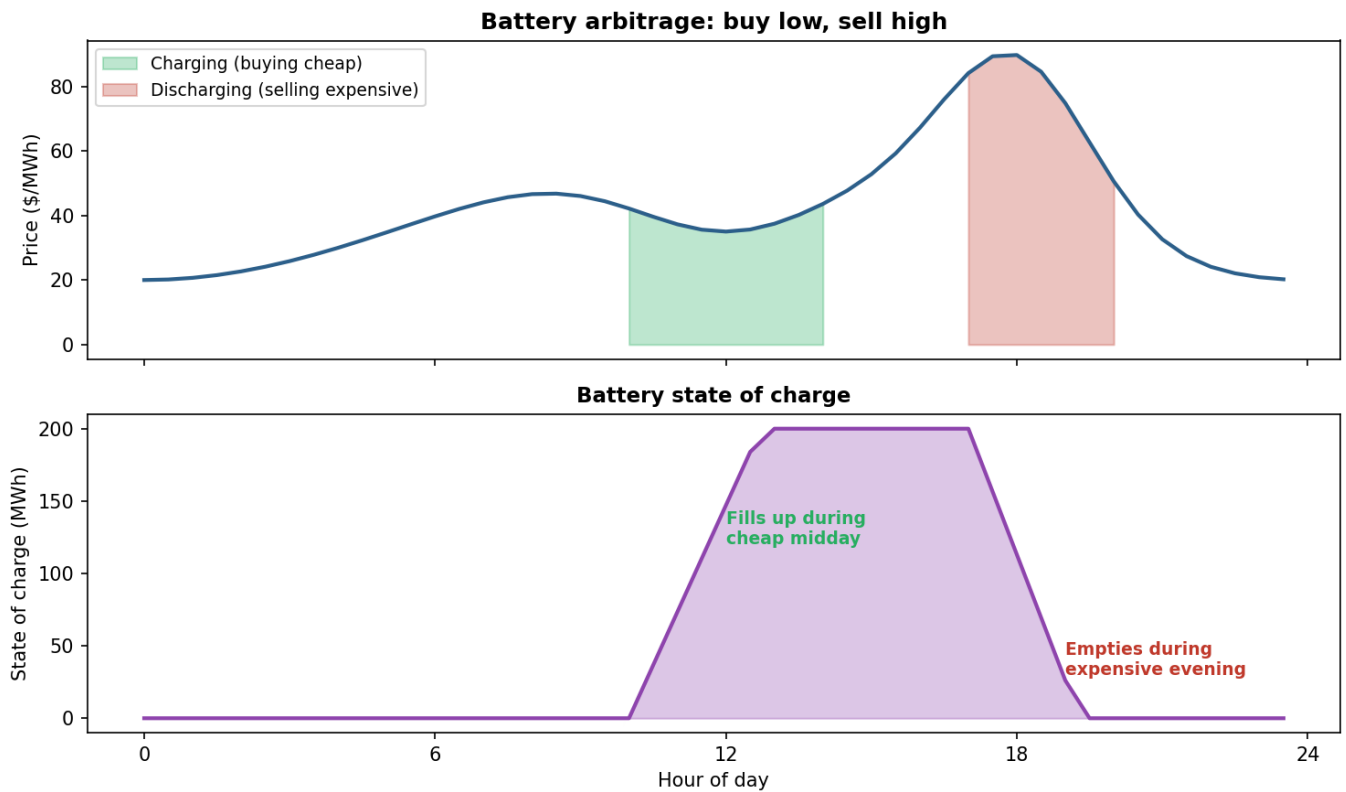


Figure 5.2 — Optimal battery dispatch from the LP. The top panel shows the price forecast (blue) and actual prices (grey). The bottom panel shows the battery's state of charge (green), with charging during low prices and discharging during high prices. The LP jointly optimises all 48 periods simultaneously.

## Why an LP, Not Heuristic Rules?

A natural instinct is to use simple rules: "charge whenever the price is below \$30, discharge whenever it is above \$100." This is intuitive but leaves money on the table:

**Rules ignore constraints.** If you charge aggressively during a moderate dip, you may be full when an even cheaper period arrives later. The LP considers all 48 periods simultaneously and plans ahead.

**Rules ignore the cycle limit.** If you cycle early in the day on modest spreads, you may exhaust your cycle budget before the biggest spread of the day. The LP allocates cycles to the most profitable spreads.

**Rules ignore energy coupling.** The charge and discharge decisions are coupled through the state of charge — you cannot discharge what you have not first charged. The LP respects this coupling exactly.

**Threshold selection is arbitrary.** What is the "right" charge threshold — \$30? \$20? \$50? It depends on the distribution of prices that day. The LP discovers the optimal thresholds implicitly.

**The LP is the gold standard:** For a given price forecast, the LP finds the mathematically optimal dispatch schedule — no heuristic can do better. This is why we formulate dispatch as an LP: it removes dispatch quality as a variable, allowing us to isolate the effect of forecast quality. If two models produce different revenues through the LP, the difference is entirely due to forecast quality, not dispatch strategy.

## Solving the LP with cvxpy

The LP is solved using **cvxpy**, a Python library for specifying and solving convex optimisation problems.

**cvxpy:** A Python-embedded modelling language for convex optimisation problems. The user specifies the objective function, variables, and constraints using natural mathematical syntax, and cvxpy translates the problem into a standard form that can be solved by one of several numerical solvers (CLARABEL, ECOS, SCS). For the battery dispatch LP with 48 time steps, the solve time is typically 1–10 milliseconds.

**Convex optimisation:** A class of optimisation problems where the objective function is convex (bowl-shaped) and the feasible region (defined by constraints) is also convex. The critical property of convex problems is that any local optimum is guaranteed to be the global optimum — the solver cannot get "stuck" in a sub-optimal solution. Linear programs are a special case of convex optimisation, so all LPs are convex.

## Model Predictive Control (MPC)

### From One-Shot to Rolling Dispatch

The LP formulation described above solves for an optimal schedule over a single 48-period window. In practice, the battery does not commit to this entire schedule at once. Instead, it uses **Model Predictive Control (MPC)** — a rolling strategy that re-optimises at every decision point.

**Model Predictive Control (MPC):** A control strategy that repeatedly solves an optimisation problem over a finite horizon, executes only the first step, then shifts the horizon forward and re-solves with updated information. MPC bridges the gap between forecasting (which looks forward) and control (which acts now). The key insight is that later actions in the schedule can be revised as better information arrives, so there is no need to commit to them in advance.

The MPC loop:

1. At the current decision point, generate a price forecast for the next H periods.
2. Solve the LP to find the optimal schedule over the full horizon.
3. **Execute only the first period's action** — charge, discharge, or do nothing.
4. Advance time by one period (30 minutes).
5. Observe the actual price for the period just completed.
6. Update the price forecast with any new information that has arrived.
7. Repeat from step 2.

### Why Only Execute the First Period?

The key insight of MPC is that **forecasts improve over time**. The forecast made at noon for 8pm is worse than the forecast made at 7:30pm for 8pm — the later forecast has access to more recent data and a shorter horizon.

**Open-loop control:** Executing a pre-computed plan without adjusting for new information. The full 48-period schedule from a single LP solve is open-loop — it commits to actions based on a single forecast, ignoring everything that happens afterward.

**Closed-loop control:** Adjusting actions based on new information as it becomes available. MPC is closed-loop — it re-optimises at each step, incorporating the latest observations and updated forecasts. Closed-loop control is always at least as good as open-loop control (and usually better), because it can never be harmful to use better information.

By re-optimising at each step, MPC captures the value of the improving forecast. If the noon forecast predicted a \$200 spike at 6pm but by 5pm the updated forecast shows only \$80, MPC can cancel the planned discharge and save the stored energy for a better opportunity. Open-loop control would have committed to discharging at \$80 based on the stale \$200 forecast.

**Real-world example — MPC saves the day:** At noon, the forecast predicts moderate prices all evening (no spike). The LP plans minimal dispatch — just one moderate charge/discharge cycle. But at 4pm, an unexpected generator trip removes 500 MW from the grid. The updated price forecast now shows a \$3,000 spike at 6pm. MPC, re-solving at 4pm, immediately plans a full charge followed by discharge into the spike. Open-loop control, locked into the noon plan, would have missed this opportunity entirely. The MPC advantage in this scenario could be worth tens of thousands of dollars.

### MPC Limitations with Point Forecasts

MPC with a point forecast (the median or expected price) treats the forecast as **certain**. It dispatches the same way whether the forecast is confident or uncertain:

- If the forecast says "6pm price will be \$200" with high confidence (narrow prediction interval), the battery should discharge aggressively — the spike is very likely.
- If the forecast says "6pm price will be \$200" with low confidence (wide prediction interval), the battery should be more conservative — the spike might not materialise, and discharging into a \$50 market wastes a cycle.

Point-forecast MPC cannot distinguish these two situations. It dispatches identically in both cases. This limitation motivates the **probabilistic dispatch** methods covered in Chapters 8 and 9, where the full distribution of possible prices informs the dispatch decision.

## The Capture Ratio

### Definition and Interpretation

The **capture ratio** is the headline economic metric that connects forecast quality to dollar value:

**Capture ratio:** The ratio of revenue earned by forecast-driven battery dispatch to the revenue that would be earned with perfect foresight (knowing the actual prices in advance). It measures how much of the theoretically available arbitrage revenue the forecast enables the battery to capture.

Capture Ratio = Revenue(forecast dispatch) / Revenue(perfect foresight dispatch)

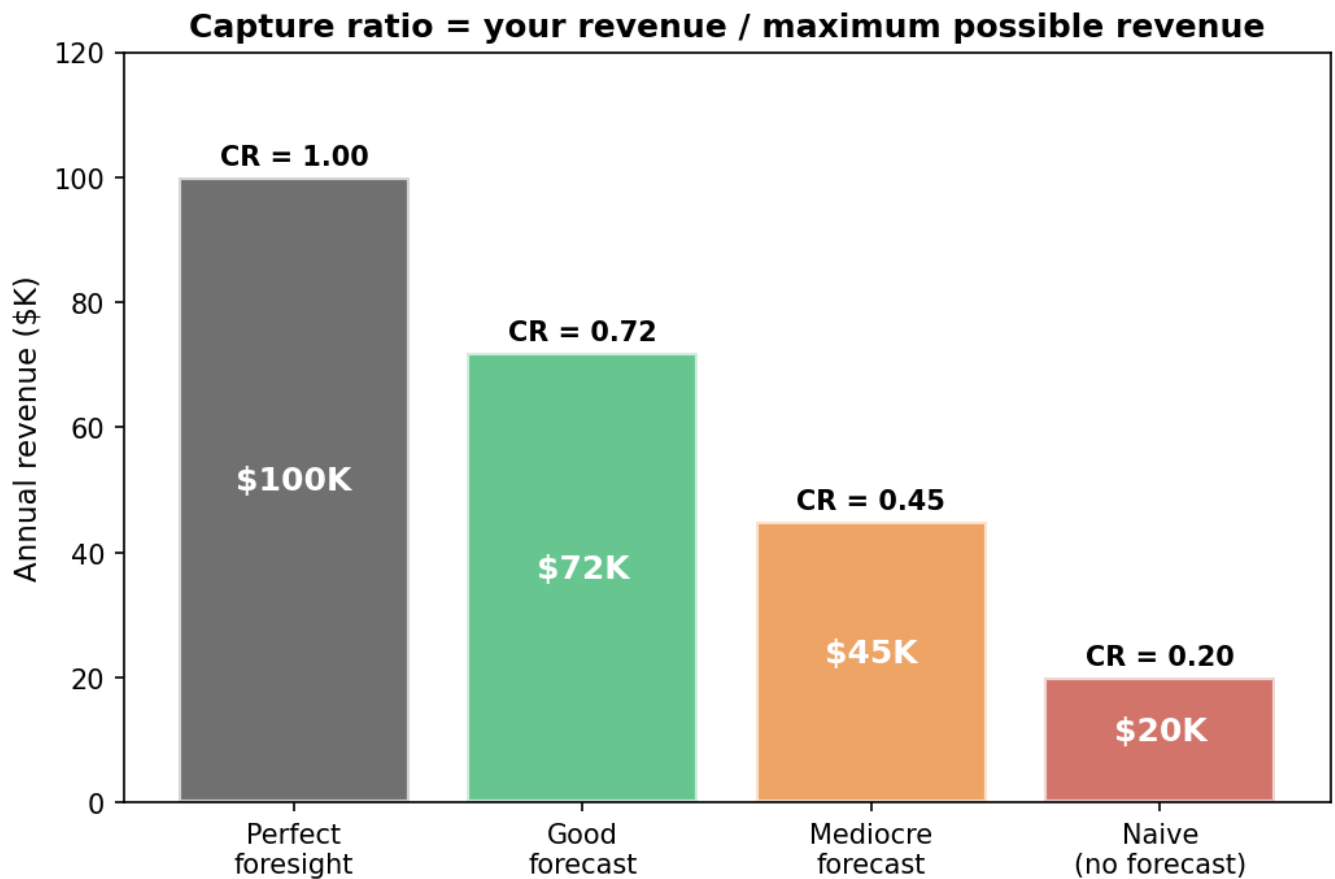


Figure 5.3 — The capture ratio measures what fraction of the perfect-foresight revenue a forecast-driven battery actually earns. Perfect foresight (CR = 1.0) is unachievable; the gap from 1.0 represents the cost of forecast imperfection.

Interpreting the capture ratio:

| Capture Ratio  | Interpretation                                                                                         |
|----------------|--------------------------------------------------------------------------------------------------------|
| CR = 1.0       | Perfect forecast — the battery captures all available revenue. Unachievable in practice.               |
| CR = 0.65–0.75 | Good forecast — the target range for a well-tuned model. This is what we aim for.                      |
| CR = 0.50      | Roughly the market average against perfect foresight — the bar your model must clear.                  |
| CR = 0.3–0.50  | Below average — significant revenue left on the table. Room for improvement.                           |
| CR = 0.0       | The forecast is useless — the battery neither makes nor loses money. Equivalent to not operating.      |
| CR < 0.0       | The forecast is worse than useless — the battery systematically buys high and sells low, losing money. |

**Perfect foresight (hindsight optimum):** The theoretical dispatch schedule that would be chosen if the actual future prices were known in advance. It represents the absolute upper bound on battery revenue for a given price trajectory. It is computed by solving the LP with *actual* prices instead of forecast prices. No real-world system can achieve perfect foresight, but it provides the denominator for the capture ratio.

## Why Capture Ratio, Not MAE?

MAE measures forecast *accuracy*\* — how close the predicted prices are to the actual prices. The capture ratio measures forecast *value*\* — how much money the forecast makes. These are related but not the same.

Consider two forecasts:

- **Forecast A:** Very accurate during quiet periods (\$40–\$80 prices), but misses every spike. MAE = \$12.
- **Forecast B:** Slightly worse during quiet periods, but correctly identifies the timing of spikes (though not their exact magnitude). MAE = \$14.

Forecast A has a better MAE, but Forecast B has a better capture ratio — because the spikes are where the money is. During quiet periods (price spread of \$20–\$40), the battery earns modest revenue regardless of forecast quality. During spikes (price spread of \$200–\$10,000), forecast quality makes the difference between capturing a windfall and missing it entirely.

**The capture ratio as the headline metric:** Throughout the remaining chapters, every model improvement is evaluated through its effect on the capture ratio. This forces us to focus on what matters economically: not the average prediction error, but the quality of dispatch decisions at the moments that drive revenue. A 5% improvement in MAE from better predictions during quiet periods is worth less than a 2% improvement from better spike timing — the capture ratio reveals this directly.

## AEMO Pre-Dispatch as Benchmark

AEMO publishes its own price forecasts through the pre-dispatch process. These are the **market's consensus forecast** — they incorporate AEMO's demand model, renewable generation forecasts, generator bids, and network constraints.

**AEMO pre-dispatch price:** AEMO's projection of the spot price for each half-hour of the upcoming trading day, computed every 30 minutes by running the dispatch engine with forecast inputs. The pre-dispatch price reflects the market's collective expectations (through generator bids) and AEMO's demand/renewable forecasts. It is the natural benchmark for any price forecasting model — beating AEMO's own forecast represents genuine added value.

Using AEMO pre-dispatch as a benchmark is informative because:

1. It is a **real, operationally-produced forecast** — not an academic exercise.
2. It incorporates information (generator bids, network constraints) that our statistical models do not have.
3. It is the forecast that **market participants actually see** — other batteries and traders react to it.
4. Beating it demonstrates that statistical/ML methods add value beyond what AEMO's engineering approach provides.

In practice, good statistical models achieve capture ratios 5–15 percentage points above AEMO pre-dispatch MPC, primarily by better predicting the *distribution* of prices (probabilistic forecasts) rather than just the point estimate. Throughout this course, we set **0.50 as the bar** (roughly the market average against perfect foresight) and **0.65–0.75 as the target range** for a well-tuned model. Every model is reported against AEMO pre-dispatch via NEMSEER.

## Glossary

| Term                    | Definition                                                          |
|-------------------------|---------------------------------------------------------------------|
| Gate closure            | Deadline by which a forecast must be issued (noon in our setup)     |
| Forecast horizon        | Time span covered by the forecast (48 half-hours = 24 hours)        |
| Point forecast          | Single predicted value per time step                                |
| Probabilistic forecast  | Prediction of quantiles or a full distribution per time step        |
| Trading day             | NEM day from 04:00 to 04:00 AEST                                    |
| Feature (predictor)     | Input variable used by a forecasting model                          |
| Autoregressive feature  | Feature constructed from past values of the target itself           |
| Net load                | Demand minus variable renewable generation                          |
| Data leakage            | Contamination of model inputs with future information               |
| Rolling-origin backtest | Walk-forward evaluation across multiple train/test splits           |
| Embargo                 | Gap between training data and forecast to prevent leakage           |
| Expanding window        | Training window that grows over time                                |
| Sliding window          | Training window of fixed length that moves forward                  |
| Similar-day naive       | Forecast using same half-hour, same day of week, one week prior     |
| MAE                     | Mean Absolute Error — average of absolute forecast errors           |
| RMSE                    | Root Mean Square Error — penalises large errors more                |
| rMAE                    | Relative MAE — model MAE divided by naive MAE                       |
| Diebold-Mariano test    | Statistical test for significance of forecast accuracy differences  |
| HAC standard error      | Standard error corrected for autocorrelation and heteroscedasticity |
| Temporal arbitrage      | Exploiting price differences across time via storage                |
| Round-trip efficiency   | Fraction of stored energy recoverable on discharge                  |
| Linear program (LP)     | Optimisation with linear objective and constraints                  |
| SOC                     | State of charge — current stored energy in the battery              |
| cvxpy                   | Python library for convex optimisation                              |
| MPC                     | Model Predictive Control — rolling re-optimisation strategy         |

|                            |                                                 |
|----------------------------|-------------------------------------------------|
| <b>Open-loop control</b>   | Executing a fixed plan without adjustment       |
| <b>Closed-loop control</b> | Adjusting plan as new information arrives       |
| <b>Capture ratio</b>       | Forecast revenue / perfect foresight revenue    |
| <b>Perfect foresight</b>   | Theoretical optimum with known future prices    |
| <b>Pre-dispatch</b>        | AEMO's projection of future dispatch and prices |

## Summary

Day-ahead electricity price forecasting is a precisely defined problem: predict 48 half-hourly trading prices using only information available before noon gate closure. The feature matrix combines autoregressive lags (exploiting the strong lag-48 autocorrelation), calendar features (encoding the duck curve and weekly patterns), weather features (proxying renewable supply and thermal demand), and engineered features like net load. Data leakage — the silent contamination of features with future information — is the most dangerous error in time-series forecasting, producing optimistic backtests that collapse in production. The rolling-origin backtest with an embargo gap prevents leakage and provides robust, multi-period performance estimates. Baseline models (the similar-day naive and autoregressive models) are surprisingly hard to beat and set the bar that any sophisticated model must clear. Battery dispatch is formulated as a linear program that finds the mathematically optimal charge–discharge schedule for any given price forecast, and Model Predictive Control applies this LP in a rolling fashion to exploit improving forecasts over time. The capture ratio — forecast-driven revenue divided by perfect-foresight revenue — is the headline metric that translates forecast quality into economic value, and AEMO's pre-dispatch price provides the operational benchmark that any model must beat to demonstrate real-world utility.